Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 6
Integrierter zbus Event Service

Command- und Input-Kanal zu einer prüfbaren native_sim-Anwendung verbinden

3.12

Zielarchitektur des zbus Event Service

01	`Shell → event_cmd_chan`	validiert input, pulse und burst und publiziert event_command
02	`Stimulus → event_input_chan`	Subscriber bedient den Adapter; Callback publiziert mit K_NO_WAIT
03	`Async-Listener → event stats`	verarbeitet im Threadkontext und aktualisiert Zustands- und Fehlerzähler

event status zeigt den Ablauf; event stats belegt Command- und Input-Bilanz.

Arbeitsauftrag in fünf Schritten

Integration mit vorgegebenen Command- und Event-Sollwerten

01

Kanäle verbinden

event_cmd_chan und event_input_chan aktiv

02

Stimuli ausführen

input, pulse und burst

03

Bilanzen prüfen

received, rejected, published, processed

04

Fehler provozieren

ungültig, busy, unsupported

05

Abnahme sichern

Skript und Sollwerte dokumentieren

Der vollständige Quellcode folgt separat; die Folien zeigen nur Auftrag, Prüfpunkte und Sollzustände.

Startzustand mit status und stats prüfen

ERWARTETE AUSGABE

```
event status
device: ready
sequence: idle
last_error: 0
last_input: none

event stats
cmd_received=0 cmd_rejected=0
input_published=0 processed=0
input_publish_failed=0
```

01 **native_sim bereit**

Stimulusadapter, GPIO-Callback und beide zbus-Kanäle sind verfügbar.

02 **Zähler definiert**

Keine Sequenz läuft; Command-, Input- und Fehlerzähler starten bei null.

status und stats dokumentieren gemeinsam den reproduzierbaren Ausgangszustand.

Wiederholbares Shell-Abnahmeskript

KOMMANDOSEQUENZ

```
event status
event stats

event pulse 20
event stats

event pulse 20
event pulse 20
event stats

event burst 4 50
event stats
```

01 Einzelszenario

Nach einem Puls: received=1, published=1 und processed=1.

02 Kombiszenario

Nach zwei weiteren Pulsen und burst 4: received=4, published=7, processed=7.

Die Sollwerte gelten bei leerem Startzustand und vereinbarter Event-Semantik.

zbus-Zähler und prüfbare Invarianten

COMMANDS

cmd_received

steigt pro syntaktisch gültigem Command

cmd_rejected zählt busy und unsupported

Shell-Fehler werden nicht publiziert

INPUTS

input_published

steigt pro erzeugtem event_input

processed steigt im Async-Listener

Sollbilanz: published == processed

FEHLER

input_publish_failed

zbus-Publish liefert einen Fehler

last_error speichert die Herkunft

kein stiller Verlust

Jeder empfangene Command und jedes Input-Event endet in einem dokumentierten Ergebnis.

Parallelität und Fehlerfälle

01	`Sequenz läuft`	event status zeigt active; der zweite Command erhöht cmd_received und cmd_rejected
02	`Parameter ungültig`	Shell-Validierung verhindert den Publish; alle zbus-Zähler bleiben unverändert
03	`Target nicht unterstützt`	optionale Devkits erhöhen received und rejected; Input-Zähler bleiben unverändert

Kein Fehlerfall darf blockieren oder einen teilweise gestarteten zbus-Ablauf hinterlassen.

Abnahmematrix für native_sim

Startzustand	device ready, sequence idle und alle zbus-Zähler auf null
Einzelpuls	+1 cmd_received, +1 input_published und +1 processed
Zwei Pulse	+2 cmd_received, +2 input_published und +2 processed
Begrenzter Burst	+1 cmd_received, +4 input_published und +4 processed
Fehler und Parallelität	ungültig: unverändert; busy: +1 received und +1 rejected

Optional: nRF52-Dongle und ESP32-C5-DevKitC-1 erst nach bestandener native_sim-Abnahme vergleichen.

Abnahme des integrierten zbus Event Service

INTEGRATION

zbus-Pfade vollständig

Shell publiziert Commands

Subscriber steuert Adapter

Callback publiziert Input

Async-Listener verarbeitet

BEOBACHTBARKEIT

Bilanz erklärbar

status zeigt Sequenz

stats zeigt vier Zähler

last_error zeigt Herkunft

Zeitmarken nachvollziehbar

ROBUSTHEIT

Alle Fälle definiert

ungültig vor Publish

busy wird rejected

Publish-Fehler gezählt

Devkits optional

Bestanden: Command- und Input-Bilanzen stimmen; native_sim bleibt reproduzierbar.